Gotchas and Pitfalls

↑ Back to top

Introduction

SymPy runs under the Python Programming Language, so there are some things that may behave differently than they do in other, independent computer algebra systems like Maple or Mathematica. These are some of the gotchas and pitfalls that you may encounter when using SymPy. See also the introductory tutorial, the remainder of the SymPy Docs, and the official Python Tutorial.

If you are already familiar with C or Java, you might also want to look at this 4 minute Python tutorial.

Ignore `#doctest: +SKIP` in the examples. That has to do with internal testing of the examples.

Equals Signs (=)

Single Equals Sign

The equals sign (`=`) is the assignment operator, not equality. If you want to do $x = y$, use `Eq(x, y)` for equality. Alternatively, all expressions are assumed to equal zero, so you can just subtract one side and use `x - y`.

The proper use of the equals sign is to assign expressions to variables.

For example:

```
>>> from sympy.abc import x, y
>>> a = x - y
>>> print(a)
x - y
```

Double Equals Signs

Double equals signs (`==`) are used to test equality. However, this tests expressions exactly, not symbolically. For example:

```
>>> (x + 1)**2 == x**2 + 2*x + 1
False
>>> (x + 1)**2 == (x + 1)**2
True
```

If you want to test for symbolic equality, one way is to subtract one expression from the other and run it through functions like `expand()`, `simplify()`, and `trigsimp()` and see if the equation reduces to 0.

```
>>> from sympy import simplify, cos, sin, expand
>>> simplify((x + 1)**2 - (x**2 + 2*x + 1))
0
>>> eq = sin(2*x) - 2*sin(x)*cos(
>>> simplify(eq)
0
>>> expand(eq, trig=True)
0
```

↑ Back to top

Note

See also Structural Equality in the Glossary.

Variables

Variables Assignment does not Create a Relation Between Expressions

When you use `=` to do assignment, remember that in Python, as in most programming languages, the variable does not change if you change the value you assigned to it. The equations you are typing use the values present at the time of creation to "fill in" values, just like regular Python definitions. They are not altered by changes made afterwards. Consider the following:

```
>>> from sympy import Symbol
>>> a = Symbol('a')  # Symbol, `a`, stored as variable "a"
>>> b = a + 1        # an expression involving `a` stored as variable "b"
>>> print(b)
a + 1
>>> a = 4            # "a" now points to literal integer 4, not Symbol('a')
>>> print(a)
4
>>> print(b)          # "b" is still pointing at the expression involving `a`
a + 1
```

Changing quantity `a` does not change `b`; you are not working with a set of simultaneous equations. It might be helpful to remember that the string that gets printed when you print a variable referring to a SymPy object is the string that was given to it when it was created; that string does not have to be the same as the variable that you assign it to.

```
>>> from sympy import var
>>> r, t, d = var('rate time short_life')
>>> d = r*t
>>> print(d)
rate*time
>>> r = 80
>>> t = 2
>>> print(d)        # We haven't changed d, only r and t
rate*time
>>> d = r*t
>>> print(d)        # Now d is using the current values of r and t
160
```

↑ Back to top

If you need variables that have dependence on each other, you can define functions. Use the `def` operator. Indent the body of the function. See the Python docs for more information on defining functions.

```
>>> c, d = var('c d')
>>> print(c)
c
>>> print(d)
d
>>> def ctimesd():
...     """
...     This function returns whatever c is times whatever d is.
...     """
...     return c*d
...
>>> ctimesd()
c*d
>>> c = 2
>>> print(c)
2
>>> ctimesd()
2*d
```

If you define a circular relationship, you will get a `RuntimeError`.

```
>>> def a():
...     return b()
...
>>> def b():
...     return a()
...
>>> a()
Traceback (most recent call last):
  File "...", line ..., in ...
    compileflags, 1) in test.globs
  File "<...>", line 1, in <module>
    a()
  File "<...>", line 2, in a
    return b()
  File "<...>", line 2, in b
    return a()
  File "<...>", line 2, in a
    return b()
...
RuntimeError: maximum recursion depth exceeded
```

↑ Back to top

> **Note**
>
> See also immutable in the Glossary.

Symbols

Symbols are variables, and like all other variables, they need to be assigned before you can use them. For example:

```
>>> import sympy
>>> z**2  # z is not defined yet
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'z' is not defined
>>> sympy.var('z')  # This is the easiest way to define z as a standard symbol
z
>>> z**2
z**2
```

If you use **isympy**, it runs the following commands for you, giving you some default Symbols and Functions.

```
>>> from __future__ import division
>>> from sympy import *
>>> x, y, z, t = symbols('x y z t')
>>> k, m, n = symbols('k m n', integer=True)
>>> f, g, h = symbols('f g h', cls=Function)
```

You can also import common symbol names from `sympy.abc`.

```
>>> from sympy.abc import w
>>> w
w
>>> import sympy
>>> dir(sympy.abc)
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O',
'P', 'Q', 'R', 'S', 'Symbol', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',
'__builtins__', '__doc__', '__file__', '__name__', '__package__', '_greek',
'_latin', 'a', 'alpha', 'b', 'beta', 'c', 'chi', 'd', 'delta', 'e',
'epsilon', 'eta', 'f', 'g', 'gamma', 'h', 'i', 'iota', 'j', 'k', 'kappa',
'l', 'm', 'mu', 'n', 'nu', 'o', 'omega', 'omicron', 'p', 'phi', 'pi',
'psi', 'q', 'r', 'rho', 's', 'sigma', 't', 'tau', 'theta', 'u', 'upsilon',
'v', 'w', 'x', 'xi', 'y', 'z', 'zeta']
```

↑ Back to top

If you want control over the assumptions of the variables, use `Symbol` and `symbols()`. See Keyword Arguments below.

Lastly, it is recommended that you not use `I`, `E`, `S`, `N`, `C`, `O`, or `Q` for variable or symbol names, as those are used for the imaginary unit (i), the base of the natural logarithm (e), the `sympify()` function (see Symbolic Expressions below), numeric evaluation (`N()` is equivalent to evalf()), the big O order symbol (as in $O(n \log n)$), and the assumptions object that holds a list of supported ask keys (such as `Q.real`), respectively. You can use the mnemonic `OSINEQ` to remember what Symbols are defined by default in SymPy. Or better yet, always use lowercase letters for Symbol names. Python will not prevent you from overriding default SymPy names or functions, so be careful.

```
>>> cos(pi)  # cos and pi are a built-in sympy names.
-1
>>> pi = 3   # Notice that there is no warning for overriding pi.
>>> cos(pi)
cos(3)
>>> def cos(x):  # No warning for overriding built-in functions either.
...     return 5*x
...
>>> cos(pi)
15
>>> from sympy import cos  # reimport to restore normal behavior
```

To get a full list of all default names in SymPy do:

```
>>> import sympy
>>> dir(sympy)
# A big list of all default sympy names and functions follows.
# Ignore everything that starts and ends with __.
```

If you have IPython installed and use **isympy**, you can also press the TAB key to get a list of all built-in names and to autocomplete. Also, see this page for a trick for getting tab completion in the regular Python console.

> **Note**
>
> See also the Defining Symbols section of the Best Practices page.

↑ Back to top

Functions

A function like `f(x)` can be created by defining the Function and the variable:

```
>>> from sympy import Function
>>> f = Function('f')
>>> x = Symbol('x')
>>> f(x)
f(x)
```

If you assign `f(x)` to a Python variable f you will lose your ability to copy and paste that function or to create a function with a different argument: `Function('f')` is callable, but `Function('f')(x)` is not:

```
>>> f1 = Function('f1')
>>> f2 = Function('f2')('x')
>>> f1
f1
>>> f2
f2(x)
>>> f1(1)
f1(1)
>>> f2(1)
Traceback (most recent call last):
...
TypeError: 'f2' object is not callable
>>> f2.subs(x, 1)
f2(1)
```

Symbolic Expressions

Python numbers vs. SymPy Numbers

SymPy uses its own classes for integers, rational numbers, and floating point numbers instead of the default Python `int` and `float` types because it allows for more control. But you have to be careful. If you type an expression that just has numbers in it, it will default to a Python expression. Use the `sympify()` function, or just `S`, to ensure that something is a SymPy expression.

```
>>> 6.2  # Python float. Notice the floating point accuracy problems.
6.2000000000000002
>>> type(6.2)  # <type 'float'> in Python 2.x,  <class 'float'> in Py3k
<... 'float'>
>>> S(6.2)  # SymPy Float has no such problems because of arbitrary precision.
6.20000000000000
>>> type(S(6.2))
<class 'sympy.core.numbers.Float'>
```

If you include numbers in a SymPy expression, they will be sympified automatically, but there is one gotcha you should be aware of. If you do `<number>/<number>` inside of a SymPy expression, Python will evaluate the two numbers bef[illegible] a chance to get to them. The solution is to `sympify()` one of the numbers, or use `Rational`.

↑ Back to top

```
>>> x**(1/2)  # evaluates to x**0 or x**0.5
x**0.5
>>> x**(S(1)/2)  # sympyify one of the ints
sqrt(x)
>>> x**Rational(1, 2)  # use the Rational class
sqrt(x)
```

With a power of `1/2` you can also use `sqrt` shorthand:

```
>>> sqrt(x) == x**Rational(1, 2)
True
```

If the two integers are not directly separated by a division sign then you don't have to worry about this problem:

```
>>> x**(2*x/3)
x**(2*x/3)
```

Note

A common mistake is copying an expression that is printed and reusing it. If the expression has a `Rational` (i.e., `<number>/<number>`) in it, you will not get the same result, obtaining the Python result for the division rather than a SymPy Rational.

```
>>> x = Symbol('x')
>>> print(solve(7*x -22, x))
[22/7]
>>> 22/7  # If we just copy and paste we get int 3 or a float
3.142857142857143
>>> # One solution is to just assign the expression to a variable
>>> # if we need to use it again.
>>> a = solve(7*x - 22, x)[0]
>>> a
22/7
```

The other solution is to put quotes around the expression and run it through S() (i.e., sympify it):

```
>>> S("22/7")
22/7
```

Also, if you do not use **isympy**, you could use `from __future__ import division` to prevent the `/` sign from performing integer division.

```
>>> from __future__ import division
>>> 1/2   # With division imported it evaluates to a python float
0.5
>>> 1//2  # You can still achieve ision with //
0
```

↑ Back to top

But be careful: you will now receive floats where you might have desired a Rational:

```
>>> x**(1/2)
x**0.5
```

`Rational` only works for number/number and is only meant for rational numbers. If you want a fraction with symbols or expressions in it, just use `/`. If you do number/expression or expression/number, then the number will automatically be converted into a SymPy Number. You only need to be careful with number/number.

```
>>> Rational(2, x)
Traceback (most recent call last):
...
TypeError: invalid input: x
>>> 2/x
2/x
```

Evaluating Expressions with Floats and Rationals

SymPy keeps track of the precision of `Float` objects. The default precision is 15 digits. When an expression involving a `Float` is evaluated, the result will be expressed to 15 digits of precision but those digits (depending on the numbers involved with the calculation) may not all be significant.

The first issue to keep in mind is how the `Float` is created: it is created with a value and a precision. The precision indicates how precise of a value to use when that `Float` (or an expression it appears in) is evaluated.

The values can be given as strings, integers, floats, or rationals.

- strings and integers are interpreted as exact

↑ Back to top

```
>>> Float(100)
100.000000000000
>>> Float('100', 5)
100.00
```

- to have the precision match the number of digits, the null string can be used for the precision

```
>>> Float(100, '')
100.
>>> Float('12.34')
12.3400000000000
>>> Float('12.34', '')
12.34
```

```
>>> s, r = [Float(j, 3) for j in ('0.25', Rational(1, 7))]
>>> for f in [s, r]:
...     print(f)
0.250
0.143
```

Next, notice that each of those values looks correct to 3 digits. But if we try to evaluate them to 20 digits, a difference will become apparent:

The 0.25 (with precision of 3) represents a number that has a non-repeating binary decimal; 1/7 is repeating in binary and decimal – it cannot be represented accurately too far past those first 3 digits (the correct decimal is a repeating 142857):

```
>>> s.n(20)
0.25000000000000000000
>>> r.n(20)
0.14285278320312500000
```

It is important to realize that although a Float is being displayed in decimal at arbitrary precision, it is actually stored in binary. Once the Float is created, its binary information is set at the given precision. The accuracy of that value cannot be subsequently changed; so 1/7, at a precision of 3 digits, can be padded with binary zeros, but these will not make it a more accurate value of 1/7.

If inexact, low-precision numbers are involved in a calculation with higher precision values, the evalf engine will increase the precision of the low precision values and inexact results will be obtained. This is feature of calculations with limited precision:

```
>>> Float('0.1', 10) + Float('0.1', 3)
0.2000061035
```

Although the `evalf` engine tried to maintain 10 digits of precision (since that was the highest precision represented) the 3-digit precision used limits the accuracy to about 4 digits – not all the digits you see are significant. evalf doesn't try to keep track of the number of significant digits.

That very simple expression involving the addition of two numbers with different precisions will hopefully be instructive in helping you understand why more complicated expressions (like trig expressions that may not be simplified) will not evaluate to an exact zero even though, with the right simplification, they should be zero. Consider this unsimplified trig identity, multiplied by a big number:

↑ Back to top

```
>>> big = 12345678901234567890
>>> big_trig_identity = big*cos(x)**2 + big*sin(x)**2 - big*1
>>> abs(big_trig_identity.subs(x, .1).n(2)) > 1000
True
```

When the $\cos$ and $\sin$ terms were evaluated to 15 digits of precision and multiplied by the big number, they gave a large number that was only precise to 15 digits (approximately) and when the 20 digit big number was subtracted the result was not zero.

There are three things that will help you obtain more precise numerical values for expressions:

1) Pass the desired substitutions with the call to evaluate. By doing the subs first, the `Float` values cannot be updated as necessary. By passing the desired substitutions with the call to evalf the ability to re-evaluate as necessary is gained and the results are impressively better:

```
>>> big_trig_identity.n(2, {x: 0.1})
-0.e-91
```

2) Use Rationals, not Floats. During the evaluation process, the Rational can be computed to an arbitrary precision while the Float, once created – at a default of 15 digits – cannot. Compare the value of `-1.4e+3` above with the nearly zero value obtained when replacing x with a Rational representing 1/10 – before the call to evaluate:

```
>>> big_trig_identity.subs(x, S('1/10')).n(2)
0.e-91
```

3) Try to simplify the expression. In this case, SymPy will recognize the trig identity and simplify it to zero so you don't even have to evaluate it numerically:

```
>>> big_trig_identity.simplify()
0
```

Immutability of Expressions

Expressions in SymPy are immutable, and cannot be modified by an in-place operation. This means that a function will always return an object, and the original expression will not be modified. The following example snippet demonstrates how this works:

```
def main():
    var('x y a b')
    expr = 3*x + 4*y
    print('original =', expr)
    expr_modified = expr.subs({x: a, y: b})
    print('modified =', expr_modified)

if __name__ == "__main__":
    main()
```

↑ Back to top

The output shows that the `subs()` function has replaced variable `x` with variable `a`, and variable `y` with variable `b`:

```
original = 3*x + 4*y
modified = 3*a + 4*b
```

The `subs()` function does not modify the original expression `expr`. Rather, a modified copy of the expression is returned. This returned object is stored in the variable `expr_modified`. Note that unlike C/C++ and other high-level languages, Python does not require you to declare a variable before it is used.

Mathematical Operators

SymPy uses the same default operators as Python. Most of these, like `*/+-`, are standard. Aside from integer division discussed in Python numbers vs. SymPy Numbers above, you should also be aware that implied multiplication is not allowed. You need to use `*` whenever you wish to multiply something. Also, to raise something to a power, use `**`, not `^` as many computer algebra systems use. Parentheses `()` change operator precedence as you would normally expect.

In **isympy**, with the **ipython** shell:

```
>>> 2x
Traceback (most recent call last):
...
SyntaxError: invalid syntax
>>> 2*x
2*x
>>> (x + 1)^2  # This is not power.  Use ** instead.
Traceback (most recent call last):
...
TypeError: unsupported operand type(s) for ^: 'Add' and 'int'
>>> (x + 1)**2
(x + 1)**2
>>> pprint(3 - x**(2*x)/(x + 1))
    2*x
   x
- ----- + 3
  x + 1
```

Inverse Trig Functions

SymPy uses different names for some functions than most computer algebra systems. In particular, the inverse trig functions use th↑ Back to top es of `asin`, `acos` and so on instead of the usual `arcsin` and `arccos`. Use the methods described in Symbols above to see the names of all SymPy functions.

Sqrt is not a Function

There is no `sqrt` function in the same way that there is an exponential function (`exp`). `sqrt(x)` is used to represent `Pow(x, S(1)/2)` so if you want to know if an expression has any square roots in it, `expr.has(sqrt)` will not work. You must look for `Pow` with an exponent of one half (or negative one half if it is in a denominator, e.g.

```
>>> (y + sqrt(x)).find(Wild('w')**S.Half)
{sqrt(x)}
>>> (y + 1/sqrt(x)).find(Wild('w')**-S.Half)
{1/sqrt(x)}
```

If you are interested in any power of the `sqrt` then the following pattern would be appropriate

```
>>> sq = lambda s: s.is_Pow and s.exp.is_Rational and s.exp.q == 2
>>> (y + sqrt(x)**3).find(sq)
{x**(3/2)}
```

Special Symbols

The symbols `[]`, `{}`, `=`, and `()` have special meanings in Python, and thus in SymPy. See the Python docs linked to above for additional information.

Lists

Square brackets `[]` denote a list. A list is a container that holds any number of different objects. A list can contain anything, including items of different types. Lists are mutable, which means that you can change the elements of a list after it has been created. You access the items of a list also using square brackets, placing them after the list or list variable. Items are numbered using the space before the item.

> **Note**
>
> List indexes begin at 0.

Example:

```
>>> a = [x, 1]  # A simple list of two items
>>> a
[x, 1]
>>> a[0]  # This is the first ite
x
>>> a[0] = 2  # You can change values of lists after they have been created
>>> print(a)
[2, 1]
>>> print(solve(x**2 + 2*x - 1, x)) # Some functions return lists
[-1 + sqrt(2), -sqrt(2) - 1]
```

↑ Back to top

> **Note**
>
> See the Python docs for more information on lists and the square bracket notation for accessing elements of a list.

Dictionaries

Curly brackets `{}` denote a dictionary, or a dict for short. A dictionary is an unordered list of non-duplicate keys and values. The syntax is `{key: value}`. You can access values of keys using square bracket notation.

```
>>> d = {'a': 1, 'b': 2}  # A dictionary.
>>> d
{'a': 1, 'b': 2}
>>> d['a']  # How to access items in a dict
1
>>> roots((x - 1)**2*(x - 2), x)  # Some functions return dicts
{1: 2, 2: 1}
>>> # Some SymPy functions return dictionaries.  For example,
>>> # roots returns a dictionary of root:multiplicity items.
>>> roots((x - 5)**2*(x + 3), x)
{-3: 1, 5: 2}
>>> # This means that the root -3 occurs once and the root 5 occurs twice.
```

> **Note**
>
> See the Python docs for more information on dictionaries.

Tuples

Parentheses `()`, aside from changing operator precedence and their use in function calls, (like `cos(x)`), are also used for tuples. A `tuple` is identical to a list, except that it is not mutable. That means that you cannot change their values after they have been created. In general, you will not need tuples in SymPy, but sometimes it can be more convenient to type parentheses instead of square brackets.

```
>>> t = (1, 2, x)  # Tuples are like lists
>>> t
(1, 2, x)
>>> t[0]
1
>>> t[0] = 4  # Except you cannot change them after they have been created
Traceback (most recent call last):
  File "<console>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

↑ Back to top

Single element tuples, unlike lists, must have a comma in them:

```
>>> (x,)
(x,)
```

Without the comma, a single expression without a comma is not a tuple:

```
>>> (x)
x
```

integrate takes a sequence as the second argument if you want to integrate with limits (and a tuple or list will work):

```
>>> integrate(x**2, (x, 0, 1))
1/3
>>> integrate(x**2, [x, 0, 1])
1/3
```

> **Note**
>
> See the Python docs for more information on tuples.

Keyword Arguments

Aside from the usage described above, equals signs (`=`) are also used to give named arguments to functions. Any function that has `key=value` in its parameters list (see below on how to find this out), then `key` is set to `value` by default. You can change the value of the key by supplying your own value using the equals sign in the function call. Also, functions that have `**` followed by a name in the parameters list (usually `**kwargs` or `**assumptions`) allow you to add any number of `key=value` pairs that you want, and they will all be evaluated according to the function.

`sqrt(x**2)` doesn't auto simplify to x because x is assumed to be complex by default, and, for example, `sqrt((-1)**2) == sqrt(1) == 1 != -1`:

↑ Back to top

```
>>> sqrt(x**2)
sqrt(x**2)
```

Giving assumptions to Symbols is an example of using the keyword argument:

```
>>> x = Symbol('x', positive=True)
```

The square root will now simplify since it knows that `x >= 0`:

```
>>> sqrt(x**2)
x
```

powsimp has a default argument of `combine='all'`:

```
>>> pprint(powsimp(x**n*x**m*y**n*y**m))
     m + n
(x*y)
```

Setting combine to the default value is the same as not setting it.

```
>>> pprint(powsimp(x**n*x**m*y**n*y**m, combine='all'))
     m + n
(x*y)
```

The non-default options are `'exp'`, which combines exponents...

```
>>> pprint(powsimp(x**n*x**m*y**n*y**m, combine='exp'))
 m + n  m + n
x     *y
```

...and 'base', which combines bases.

```
>>> pprint(powsimp(x**n*x**m*y**n*y**m, combine='base'))
     m      n
(x*y) *(x*y)
```

> **Note**
>
> See the Python docs for more information on function parameters.

Getting help from within SymPy

help()

↑ Back to top

Although all docs are available at docs.sympy.org or on the SymPy Wiki, you can also get info on functions from within the Python interpreter that runs SymPy. The easiest way to do this is to do `help(function)`, or `function?` if you are using **ipython**:

```
In [1]: help(powsimp)  # help() works everywhere

In [2]: # But in ipython, you can also use ?, which is better because it
In [3]: # it gives you more information
In [4]: powsimp?
```

These will give you the function parameters and docstring for `powsimp()`. The output will look something like this:

↑ Back to top

```
sympy.simplify.simplify.powsim…ep=False, combine='all',
    force=False, measure=<function count_ops>)                    [source]
```

Reduce expression by combining powers with similar bases and exponents.

EXPLANATION

If `deep` is `True` then powsimp() will also simplify arguments of functions. By default `deep` is set to `False`.

If `force` is `True` then bases will be combined without checking for assumptions, e.g. sqrt(x)*sqrt(y) -> sqrt(x*y) which is not true if x and y are both negative.

You can make powsimp() only combine bases or only combine exponents by changing combine='base' or combine='exp'. By default, combine='all', which does both. combine='base' will only combine:

```
 a   a          a                               2x      x
x * y  =>  (x*y)   as well as things like 2   =>  4
```

and combine='exp' will only combine

```
 a   b       (a + b)
x * x  =>  x
```

combine='exp' will strictly only combine exponents in the way that used to be automatic. Also use deep=True if you need the old behavior.

When combine='all', 'exp' is evaluated first. Consider the first example below for when there could be an ambiguity relating to this. This is done so things like the second example can be completely combined. If you want 'base' combined first, do something like powsimp(powsimp(expr, combine='base'), combine='exp').

EXAMPLES

```
>>> from sympy import powsimp, exp, log, symbols
>>> from sympy.abc import x, y, z, n
>>> powsimp(x**y*x**z*y**z, combine='all')
x**(y + z)*y**z
>>> powsimp(x**y*x**z*y**z, combine='exp')
x**(y + z)*y**z
>>> powsimp(x**y*x**z*y**z, combine='base', force=True)
x**y*(x*y)**z
```

```
>>> powsimp(x**z*x**y*n**z*n**y, combine='all', force=True)
(n*x)**(y + z)
>>> powsimp(x**z*x**y*n**z*n**y, combine='exp')
n**(y + z)*x**(y + z)
>>> powsimp(x**z*x**y*n**z*n**y, combine='base', force=True)
(n*x)**y*(n*x)**z
```

↑ Back to top

```
>>> x, y = symbols('x y', positive=True)
>>> powsimp(log(exp(x)*exp(y)))
log(exp(x)*exp(y))
>>> powsimp(log(exp(x)*exp(y)), deep=True)
x + y
```

Radicals with Mul bases will be combined if combine='exp'

```
>>> from sympy import sqrt
>>> x, y = symbols('x y')
```

Two radicals are automatically joined through Mul:

```
>>> a=sqrt(x*sqrt(y))
>>> a*a**3 == a**4
True
```

But if an integer power of that radical has been autoexpanded then Mul does not join the resulting factors:

```
>>> a**4 # auto expands to a Mul, no longer a Pow
x**2*y
>>> _*a # so Mul doesn't combine them
x**2*y*sqrt(x*sqrt(y))
>>> powsimp(_) # but powsimp will
(x*sqrt(y))**(5/2)
>>> powsimp(x*y*a) # but won't when doing so would violate assumptions
x*y*sqrt(x*sqrt(y))
```

Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024